

Séance 1 — Premiers programmes Python

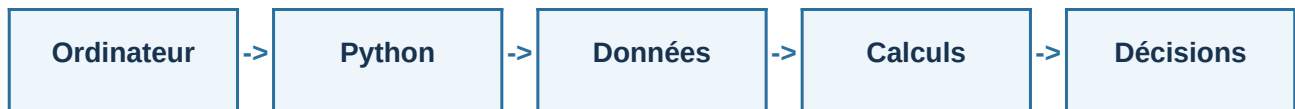
Introduction à la programmation Python — IUT 1re année

Séance 1

Premiers programmes Python

De la machine aux données, calculs et décisions

Nous commencerons dans la console Python, utilisée comme une calculatrice, puis nous enregistrerons les mêmes instructions dans un véritable script :



Le fil rouge sera une mesure de température : la saisir, la convertir, effectuer un calcul et choisir un message selon sa valeur.

À la fin de la séance, vous devez être capables de lire et d'écrire ce type de script simple.

Cette séance pose les bases utilisées dans tout le cours : exécuter un programme, représenter une donnée, effectuer un calcul, communiquer avec l'utilisateur et prendre une décision. Les séances suivantes permettront d'organiser ces traitements et de les appliquer à des séries de mesures.

Architecture matérielle

Un programme utilise plusieurs ressources de l'ordinateur :

	Élément	Rôle principal
CPU		exécuter les instructions
RAM		accueillir le programme et les données en cours d'utilisation
Stockage		conserver durablement programmes et fichiers
Périphériques		échanger avec l'extérieur



La RAM est une mémoire de travail rapide mais volatile : son contenu disparaît lorsque la machine est éteinte. Le CPU exécute des instructions machine ; dans notre cas, l'interpréteur fait le lien entre le code Python et leur exécution. Les périphériques regroupent notamment le clavier, l'écran et les capteurs.

Architecture logicielle

Le **système d'exploitation** gère la mémoire, les fichiers, les périphériques et le lancement des applications.

Dans son usage courant, un programme Python suit cette chaîne :



```
print("Première mesure")
```

On oppose souvent les programmes compilés, traduits avant leur exécution, aux programmes interprétés, pris en charge par un interpréteur. Cette opposition est un modèle simplifié : l'implémentation de référence de Python produit également une représentation intermédiaire appelée bytecode.

Console interactive et script

Python peut être utilisé de deux manières complémentaires :

Console interactive

tester rapidement une instruction
obtenir un résultat immédiat
expérimenter

Script `.py`

conserver un programme
enchaîner plusieurs traitements
modifier et réutiliser le code

```
>>> temperature = 20.5 >>> temperature + 273.15 293.65
```

Dans Thonny, le **Shell** sert à essayer une instruction ; l'éditeur sert à construire puis conserver `mesure.py`.

Le même script peut aussi être lancé avec `python mesure.py` dans un terminal (`py mesure.py` sur certaines installations Windows).

Trouver une information fiable

Personne ne mémorise toute la bibliothèque Python. Deux réflexes sont utiles :

```
help(print) help(float)
```

- `help()` décrit une fonction, un type ou une valeur disponible dans l'environnement ;
- docs.python.org est la documentation officielle du langage et de sa bibliothèque standard.

Pour une fonction externe, par exemple NumPy ou pyserial, il faut consulter la documentation de la bibliothèque concernée.

Variables et affectation

Une variable permet de donner un nom à une valeur pour la réutiliser :

```
temperature = 20.5
```

```
temperature = 20.5 ■■■■■■■■■■■■ ■■■■ nom valeur
```

= réalise une **affectation**. Une nouvelle affectation remplace la valeur associée au nom.

Une affectation n'est pas une égalité mathématique. Python évalue l'expression située à droite, puis associe son résultat au nom situé à gauche. Des noms comme `temperature` ou `nombre_mesures` rendent le programme plus compréhensible que des noms génériques comme `x` ou `a`.

Types de données

Le type décrit la nature d'une valeur et détermine les opérations possibles :

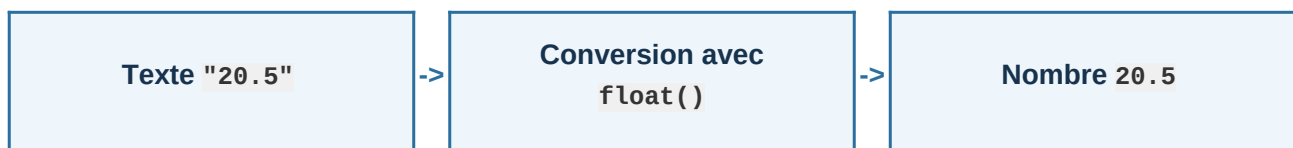
Type	Nature	Exemple
<code>int</code>	nombre entier	<code>299_792_458</code>
<code>float</code>	nombre à virgule flottante	<code>1.602e-19</code>
<code>str</code>	chaîne de caractères	<code>"COM3"</code>
<code>bool</code>	résultat logique	<code>True</code> ou <code>False</code>

```
print(type(20.5)) # <class 'float'> print(type(20 > 10)) # <class 'bool'>
```

Python utilise un typage dynamique : le type est associé à la valeur et déterminé pendant l'exécution. La fonction `type()` permet d'observer le type d'une donnée, ce qui est notamment utile pour comprendre une erreur de conversion.

Conversions explicites

Une donnée doit parfois changer de représentation avant d'être utilisée :



```
temperature = float("20.5") nombre_mesures = int("12") message = str(20.5)
```

`float()`, `int()` et `str()` produisent une nouvelle valeur du type demandé. Une conversion échoue si le contenu n'est pas compatible : `float("vingt")` provoque une erreur.

Dans un programme Python, le séparateur décimal est toujours le point : `20.5`, et non `20,5`.

Opérateurs arithmétiques

Python fournit les opérations usuelles :

Expression	Opération	Résultat pour <code>a = 7, b = 2</code>
<code>a + b</code>	addition	9
<code>a - b</code>	soustraction	5
<code>a * b</code>	multiplication	14
<code>a / b</code>	division	3.5
<code>a // b</code>	quotient arrondi vers le bas	3
<code>a % b</code>	reste	1
<code>a ** b</code>	puissance	49

Les parenthèses permettent de rendre l'ordre des calculs explicite.

L'opérateur `//` est parfois appelé « division entière », mais cette expression peut être trompeuse : le résultat est arrondi vers le bas. Ainsi, `-7 // 2` produit `-4`. Le reste `%` permet notamment de tester si un entier est pair avec la condition `n % 2 == 0`.

Affectation, comparaison et mise à jour

Il faut distinguer la modification d'une variable et la comparaison de deux valeurs :

Écriture	Rôle	Résultat
<code>temperature = 20</code>	affecter	la variable reçoit 20
<code>temperature == 20</code>	comparer	True ou False
<code>temperature != 20</code>	comparer	True ou False

Deux écritures possibles pour la même mise à jour :

```
nombre = nombre + 1 # ou, sous forme abrégée : nombre += 1
```

Autres comparaisons : `<`, `<=`, `>` et `>=`.

Précision des nombres flottants

Les nombres décimaux ne peuvent pas tous être représentés exactement en mémoire :

```
resultat = 0.1 + 0.2 print(resultat)
```

```
0.30000000000000004
```

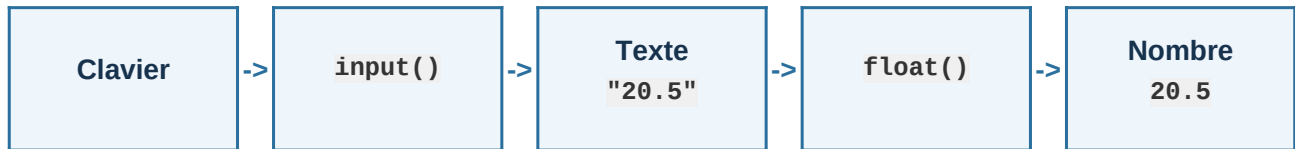
Un `float` représente donc une valeur numérique avec une précision limitée. Cette approximation est normale et doit être prise en compte lors des comparaisons.

De nombreux nombres décimaux ne possèdent pas de représentation binaire finie. Pour comparer deux résultats calculés, on utilise parfois une tolérance, par exemple `abs(a - b) < 1e-9`, plutôt qu'une égalité stricte avec `==`.

Saisie et conversion

`input()` permet de recevoir une information saisie au clavier. Son résultat est toujours une chaîne de caractères :

```
saisie = input("Température : ") temperature = float(saisie)
```



Après la conversion, `temperature` peut être utilisée dans un calcul.

`input()` interrompt le programme jusqu'à la validation de la saisie. Une saisie invalide comme vingt provoque une erreur pendant la conversion. La validation des données sera approfondie plus tard ; pour le moment, on suppose que l'utilisateur fournit une valeur compatible.

Affichage et f-strings

`print()` affiche une information. Une f-string permet d'intégrer une valeur dans un texte :

```
temperature = 20.456 print(f"Température : {temperature:.1f} °C")
```

```
Température : 20.5 °C
```

Le format `.1f` affiche la valeur arrondie à un chiffre après la virgule. Il modifie l'affichage, pas la valeur utilisée dans les calculs.

Prendre une décision

Un programme peut adapter son comportement à la valeur d'une mesure :

```
if temperature < 0: print("Risque de gel") elif temperature < 30:  
print("Température dans la plage prévue") else: print("Seuil supérieur dépassé")
```

Les conditions sont évaluées dans l'ordre. Python exécute la première branche dont la condition est vraie ; `else` traite les autres cas. L'indentation délimite les instructions de chaque branche.

`if` introduit le premier test, `elif` ajoute un test lorsque les précédents sont faux et `else` traite tous les autres cas. Une seule branche de cette structure est exécutée. Si le second test est atteint, Python sait déjà que la température est supérieure ou égale à zéro.

Combiner des conditions

Dans une condition, les opérateurs logiques permettent de combiner plusieurs tests :

	Opérateur	Signification
	<code>and</code>	les deux conditions sont vraies
	<code>or</code>	au moins une condition est vraie
	<code>not</code>	inverse une valeur logique

```
if 0 <= temperature < 30: print("Température dans la plage prévue")
```

Python autorise cette comparaison enchaînée. Pour des tests de nature différente, on utilise `and`, `or` et `not`. Si une condition devient difficile à lire, il est préférable de la décomposer.

Comprendre les erreurs

Une erreur fait partie du travail de programmation. Il faut d'abord identifier sa catégorie :

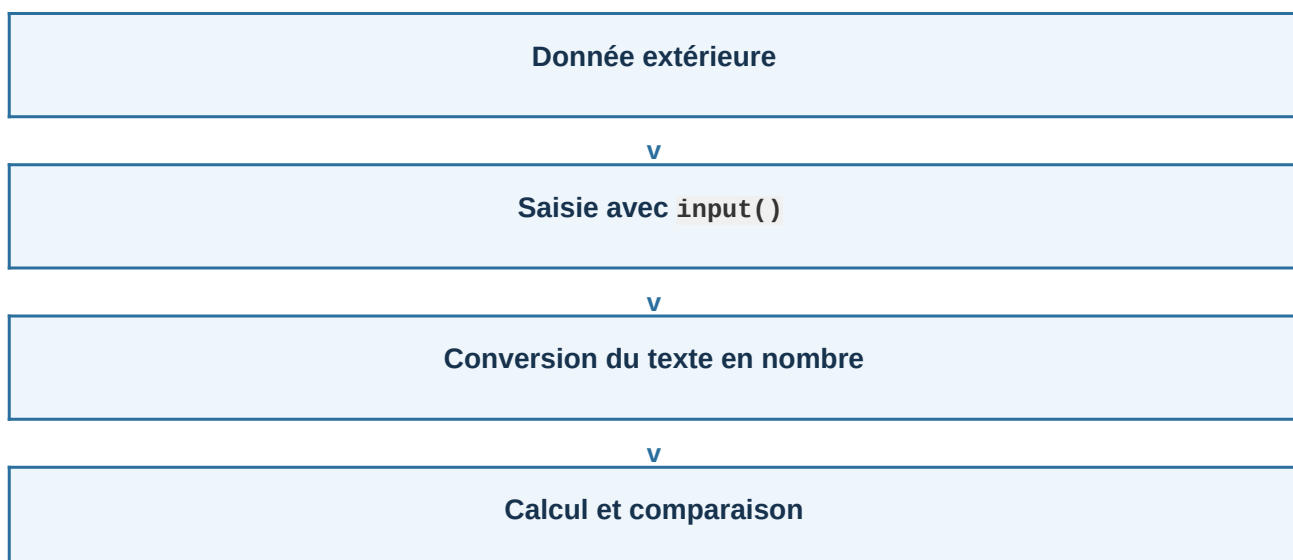
	Catégorie	Exemple
syntaxe		parenthèse ou deux-points absents
exécution		conversion impossible avec <code>float("abc")</code>
logique		seuil incorrect dans une condition

Les erreurs de syntaxe et d'exécution produisent généralement un message indiquant la ligne et la nature du problème. Une erreur de logique doit être repérée en vérifiant les résultats.

Une erreur de syntaxe empêche Python de comprendre le programme. Une erreur d'exécution apparaît pendant une opération impossible. Une erreur de logique est plus discrète : le programme s'exécute, mais produit un mauvais résultat. Dans un traceback, la dernière ligne donne généralement le type d'erreur.

Synthèse de la séance

Nous pouvons maintenant construire une première chaîne de traitement :



v

Décision avec `if`

v

Résultat affiché

Le programme sait recevoir une donnée, la convertir, la mémoriser, effectuer un calcul, prendre une décision et afficher un résultat. La séance suivante permettra d'organiser les traitements avec des fonctions et de manipuler plusieurs valeurs.

Exercices — Séance 1 : Premiers programmes Python

Ex. 1 — Premier script Créer un fichier `bonjour.py` qui affiche votre nom et votre filière, puis l'exécuter depuis un terminal.

Ex. 2 — Calculatrice d'opérations Demander deux nombres à l'utilisateur (`input`) et afficher le résultat de `+` `-` `*` `/` `//` `%` `**` entre eux, avec des f-strings lisibles. Pour ce premier exercice, choisir un second nombre non nul.

Ex. 3 — Conversions de types Demander une chaîne comme `"3.14"`, la convertir en `float`, puis afficher la valeur avec deux chiffres après la virgule à l'aide d'une f-string.

Ex. 4 — Précision flottante Calculer `0.1 + 0.2`, comparer à `0.3` avec `==` puis avec une tolérance (`abs(a-b) < 1e-9`). Expliquer la différence en commentaire.

Ex. 5 — Classification d'une mesure Demander une pression en bar et afficher : "normale" si `1 <= pression <= 2`, "alerte" si `0 <= pression < 1` ou `2 < pression <= 3`, et "danger" si `pression > 3`. Une valeur négative doit être signalée comme invalide.

Ex. 6 — Autonomie d'une batterie Demander la capacité d'une batterie en ampères-heures et le courant consommé en ampères. Calculer l'autonomie théorique `capacite / courant`, l'afficher avec une décimale, puis indiquer "faible" sous 2 h, "moyenne" de 2 h à moins de 5 h, ou "élevée" à partir de 5 h. Signaler le cas d'un courant nul ou négatif.